FOSSASIA

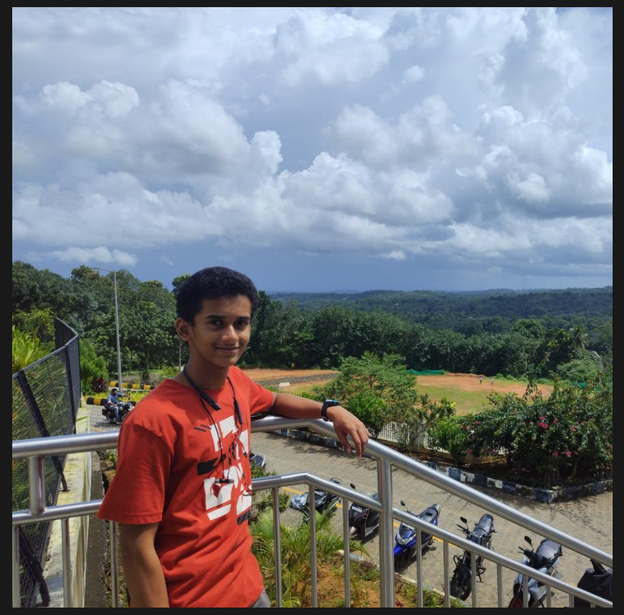
BISCUIT

DETERMINISTIC WILDCARD INDEXING IN POSTGRESQL

SIVAPRASAD MURALI

2026

FOSSASIA



About Me

Sivaprasad Murali

Student @ Indian Institute of Information Technology, Kottayam

- FOSS contributor in Python & PostgreSQL
- Interested in open-source tooling and backend systems
- Enjoys diving deep into technical details

📧 sivaprasad.off@gmail.com

FOSSASIA

% WILDCARD SEARCH

BISCUIT

```
SELECT * FROM products
WHERE pname LIKE 'PRD-____';
```

FOSSASIA

WHAT works? breaks?

Existing Methods

BTrees

Used for equality matches, range queries, and prefix searches

Trigrams

Used for fast substring, wildcard (%foo%), and fuzzy searches

Breaking Points

BTREES	Leading wildcards ('%foo', '%bar%')
PG_TRGM	Very short strings, NOT LIKE, underscores

FOSSASIA

The Gap

A DETERMINISTIC INDEX FOR ARBITRARY WILDCARD SEARCHES, INCLUDING UNDERSCORES, NEGATIONS AND CASE-INSENSITIVE SEARCHES.

FOSSASIA

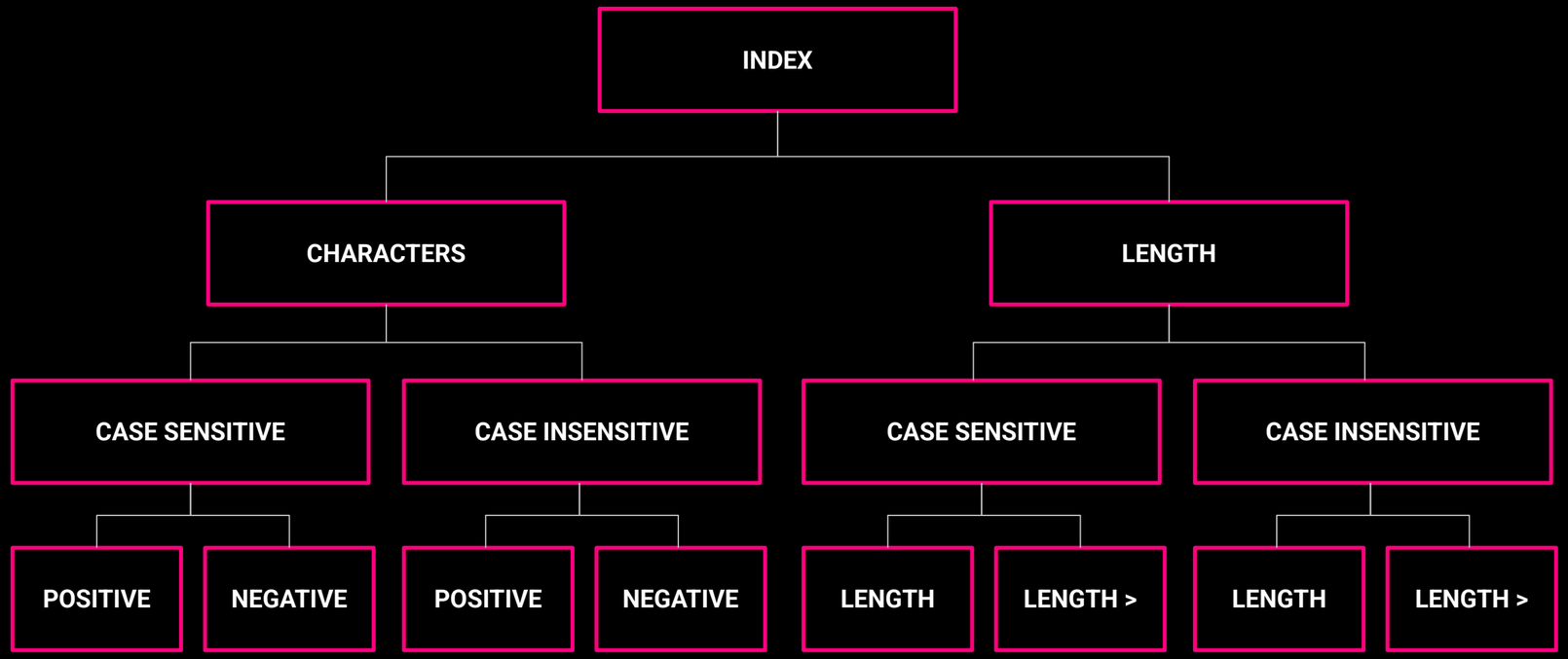
What Biscuit indexes

Characters

Length

FOSSASIA

What Biscuit indexes



FOSSASIA

Algorithm

%abc	**Negative indexing**	$char[a][-3] \wedge char[b][-2] \wedge char[c][-1]$
a_c%	**Underscore Skipping**	$char[a][0] \wedge char[c][2]$
%abc%	**Substring matching**	$\bigvee_i (char[a][i] \wedge char[b][i+1] \wedge char[c][i+2])$
%a%b%c%	**Windowed partition**	$\bigvee_i (char[a][i] \wedge \bigvee \square (char[b][j] \wedge \bigvee \square (char[c][k])))$ where $i < j < k$
abc%cba	**Length Filters**	$char[a][0] \wedge char[b][1] \wedge char[c][2] \wedge$ $char[c][-3] \wedge char[b][-2] \wedge char[a][-1] \wedge len_ge[6]$

FOSSASIA

Example

TABLE: DICTIONARY

Numbering	WORDS
1	CAMP
2	LAVA
3	WAVE
4	DIME

FOSSASIA

Example

Sample Query

```
SELECT * FROM DICTIONARY WHERE WORDS LIKE '%AVE';
```

FOSSASIA

Numbering	WORDS
1	CAMP
2	LAVA
3	WAVE
4	DIME

	1	2	3	4	
A [-3]	1	1	1	0	
V [-2]	0	1	1	0	
E [-1]	0	0	1	1	
Result	0	0	1	0	

&

&

FOSSASIA

Benchmarks

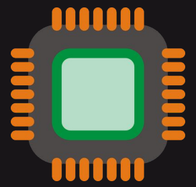
```
postgres=# explain analyse select * from test_data where email like 'user_@%';
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
 Index Scan using idx_bisc on test_data  (cost=0.25..4.27 rows=1 width=122) (actual time=0.047..0.050 rows=9 loops=1)
   Index Cond: (email ~~ 'user_@%'::text)
 Planning Time: 0.110 ms
 Execution Time: 0.069 ms
(4 rows)


postgres=# explain analyse select * from test_data where email like 'user_@%';
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on test_data  (cost=38.86..42.87 rows=1 width=122) (actual time=2.664..4.627 rows=9 loops=1)
   Recheck Cond: (email ~~ 'user_@%'::text)
   Rows Removed by Index Recheck: 9991
   Heap Blocks: exact=193
   ->  Bitmap Index Scan on idx_trgm  (cost=0.00..38.86 rows=1 width=0) (actual time=2.616..2.616 rows=10000 loops=1)
         Index Cond: (email ~~ 'user_@%'::text)
 Planning Time: 0.142 ms
 Execution Time: 4.666 ms
(8 rows)
```

email
user1@example.com user2@example.com . . user9999@example.com user10000@example.com

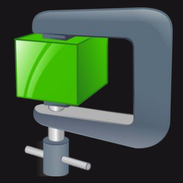
FOSSASIA

Historical hurdles

Memory



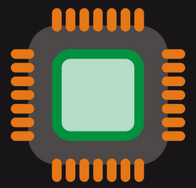
Processing



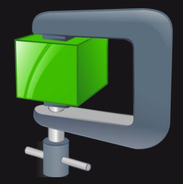
Compression

BISCUIT enablers

Cheap Memory



SIMD operations



Roaring Bitmaps

Features

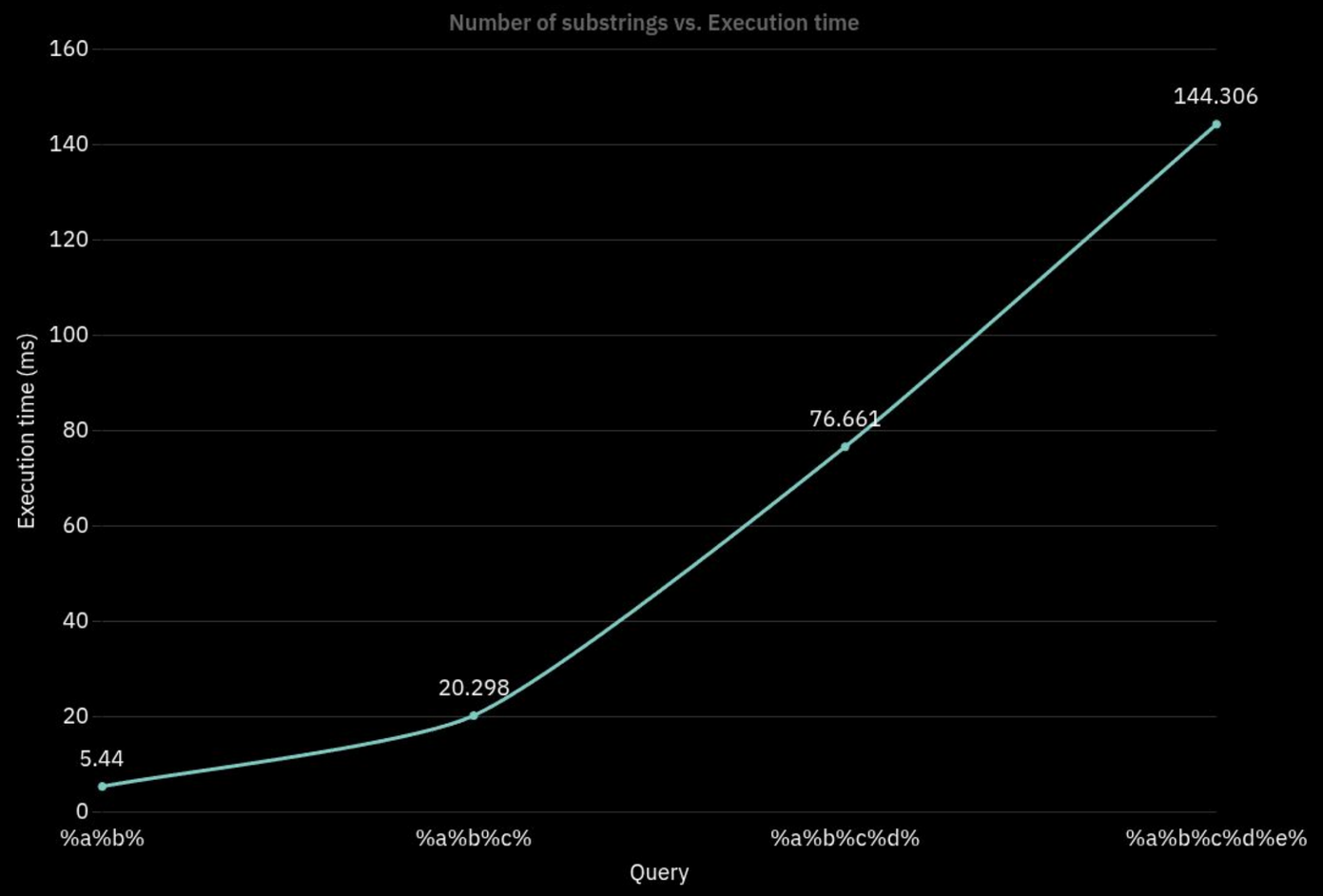
- Zero rechecks
- LIKE / ILIKE
- Arbitrary wildcards
- Negations

Challenges

- Collation indeterminism
- Index Size
- Combinatorial explosion
- Write latency

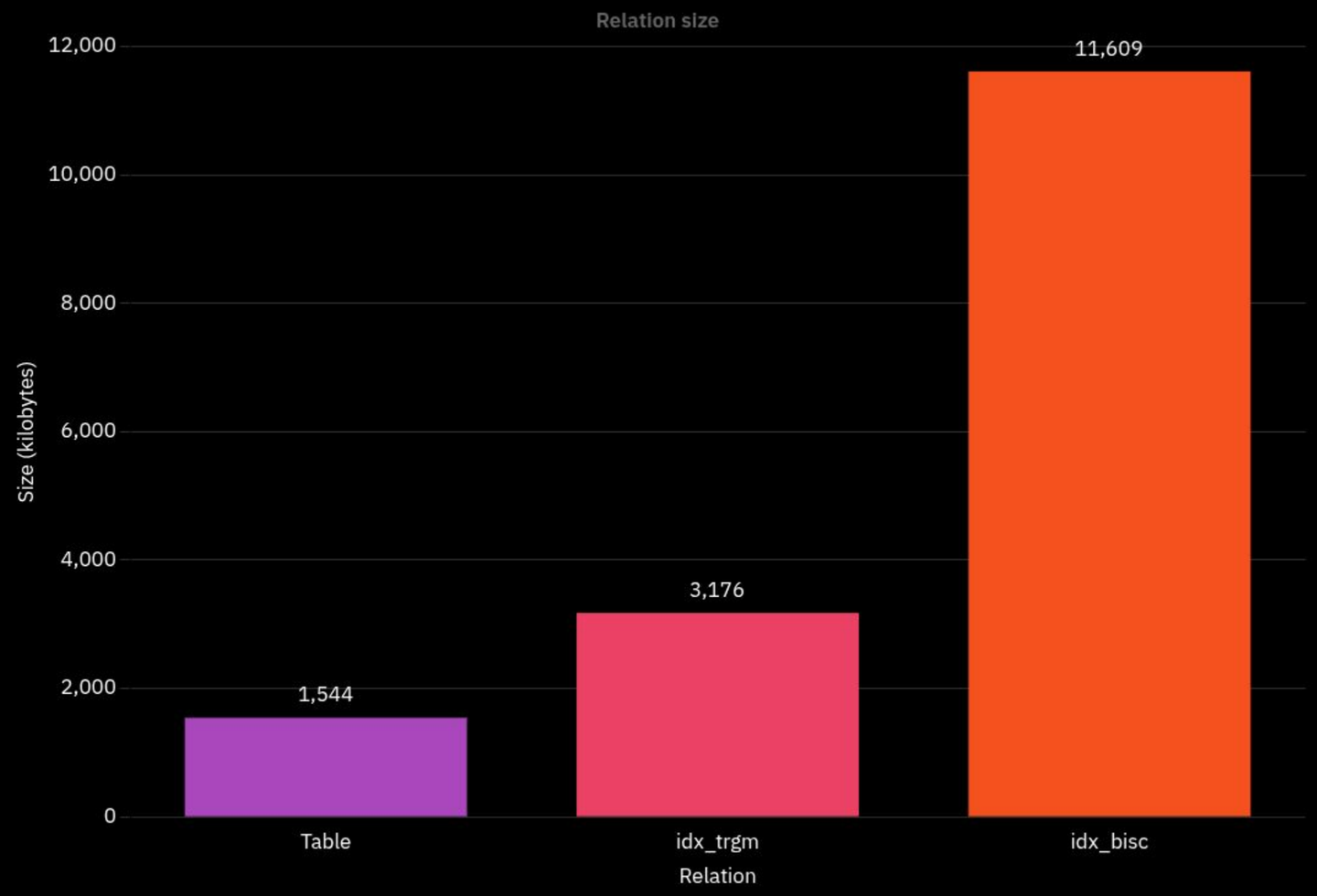
FOSSASIA

Combinatorial Explosion



FOSSASIA

Memory Size



FOSSASIA

Future Prospects

Disk Persistence

Regex Support

FOSSASIA

Try it today!

BISCUIT

```
CREATE EXTENSION BISCUIT;
CREATE INDEX ON user_table USING BISCUIT(name);
```

FOSSASIA

LinkedIn

THANK YOU!

MORE INFORMATION: WWW.BISCUIT.READTHEDOCS.IO